

NAND Prefetch 모델의 Blind spot 상세 검토

모델의 가정과 구현 가치 사이를 검증하기 위한 고찰 자료

2026년 10월 6일 | 검토 커밋 4d3bb3f | 기존 재현 기준 bf63379

현재 모델은 낮은 AXI outstanding에서 사전 준비의 가치가 크고, 충분한 outstanding에서는 demand-only도 R 채널 상한에 도달할 수 있음을 보여 준다. 다음 판단의 핵심은 그 이득이 실제로 관측 가능한 정보와 동일한 물리 자원 안에 서도 남는지다.

이 문서는 저장소 코드·103조건 CSV·기존 심화 설계검토 문서를 바탕으로 숨은 가정을 풀어 설명하고, 각 가정을 검증 가능한 설계 질문으로 바꾼다. 새로운 simulator sweep은 수행하지 않았다. 계산 예시는 이론 추정이며 후속 구현과 실험은 제안이다.

가장 먼저 정할 네 가지

우선순위	설계 질문	판단이 바뀌는 이유
P0	64KiB 요청 주소가 언제 공개되는가	알려진 demand의 조기 발행과 미래 예측의 이득을 구분한다.
P0	잘못된 예측을 언제 폐기할 수 있는가	즉시 정답 판정이 사라지면 버퍼와 경합 비용이 늘 수 있다.
P0	같은 자원이라는 말의 단위는 무엇인가	AR credit, page descriptor, SRAM bytes를 분리한다.
P1	제품에서 줄여야 할 비용은 무엇인가	같은 처리량의 credit 절감, p99, 면적, 전력 중 목표를 정한다.

자료를 읽는 순서

먼저 2장의 측정 범위를 확인하고 3~6장의 비교 기준·인과성·자원·수명주기를 결정한다. 7~15장은 예측 품질과 물리·통 계·workload 제약을 검토할 때 참고한다. 16~19장은 계측, 실험, 테스트와 구현 순서를 정하는 작업 자료다. 마지막 장에 는 근거와 코드 위치를 모았다.

표기 원칙 [측정]은 저장된 실행 결과, [코드]는 소스에서 확인한 동작, [이론]은 명시한 가정 아래의 계산, [제안]은 아직 실행하지 않은 설계와 검증 이다.

02 측정과 코드

현재 확인된 사실과 해석의 범위

현재 결과는 순차 주소, 128채널·4포트, 16KiB NAND page와 64B AXI burst를 사용한 이벤트 모델이다. 103조건의 결과는 보존되어 있지만, 인과적 online predictor나 동일 하드웨어 비용의 비교를 검증한 자료는 아니다. [R1, R4, R5]

실행 조건	전체 GB/s	후반 GB/s
기존 demand O502 128요청	13.408	13.419
기존 ready O502 128요청	252.145	256.000
기존 demand O10000 128요청	197.265	256.000
긴 demand O8000 512요청	198.598	210.769
긴 demand O10000 512요청	238.264	256.000
O10000 버퍼 2MiB 512요청	212.695	225.000
O10000 버퍼 2.5MiB 512요청	238.264	256.000

기존 128요청의 후반은 마지막 50% completion window이고, 새 실험의 late는 마지막 25%다. 같은 이름처럼 보여도 시간창 정의가 다르다. Ready의 전체 평균은 사전 준비 시간을 제외한다.

단위와 자원 점유

Outstanding O는 포트당 AR 수락부터 RLAST까지 점유하는 burst credit다. 64KiB host request는 이 구성에서 1,024 burst와 4 page로 나뉜다. 총 AR credit은 40이며, lookup slot은 lookup 완료 시, page 버퍼 예약은 해당 page의 구성 burst가 모두 반환된 후 해제된다.

현재 이름에 남아 있는 주의점

online은 미래 request ID·도착 시각을 알고 실행만 host 구간으로 제한한 합성 모델이다. same_total은 64B AR credit과 16KiB speculative page entry를 각각 1로 더한 제약이다. end_to_end_gbps는 ready lead를 포함하지만 마지막 demand 이후의 drain은 분모에서 제외한다. [R1]

56개 테스트 통과는 기존 50개와 추가 6개의 현재 모델 검사 결과다. 새로운 page 재사용, prefix 인과성, multi-ID ordering의 검증 완료를 뜻하지 않는다.

03 Blind spot 1 P0

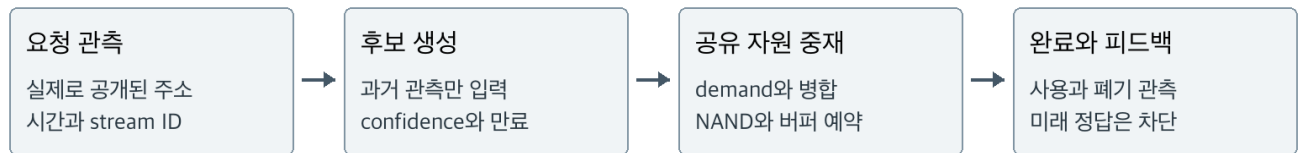
주소를 미리 아는 효과와 예측 효과

가장 큰 비교상의 빈틈은 controller가 이미 아는 정보를 demand-only 기준선이 활용하지 못할 가능성이다. 64KiB 요청 전체 주소를 알고 있는데 NAND 읽기를 각 64B AR 수락 이후까지 미루면, prefetch처럼 보이는 개선의 일부는 알려진 일을 일찍 시작한 효과일 수 있다.

현재 모델은 demand event에서 모든 구성 burst를 외부 AR queue에 넣고, lookup 완료 뒤 page를 demand로 등록한다. 이것만으로 실제 controller가 AR 이전에 주소를 아는지 알 수는 없다. 실제 IP 경계와 host queue 위치를 먼저 정해야 한다. [R2, R3]

비교군	사용 가능한 정보	분리할 효과
D0 기존 demand	수락한 AR와 완료된 lookup	기존 수치 재현
D1 알려진 demand 조기 발행	이미 공개된 64KiB 요청 주소	page read-ahead와 AR gate 해제
P 인과적 predictor	D1 정보와 과거 stream 이력	아직 공개되지 않은 주소의 예측
R oracle ready	미래 요청 목록과 준비 시간	이상적 사전 준비 참고치

D1은 제품이 해당 주소를 실제로 관측할 수 있을 때만 허용한다. 세 비교군은 같은 NAND·버퍼·page 예산·cache 정책을 사용하고, D1의 공개 요청 queue 저장 비용도 controller 경계 안이라면 계산한다.



제안 구조 미래 trace는 workload 공급기와 사후 평가기에만 둔다

그림 1 관측 정보와 사후 정답 평가를 분리하는 제안 구조

작은 실험으로 가설을 가르는 방법

동일한 순차 trace에서 D0, D1, P를 비교한다. D1이 P와 비슷한 이득을 보이면 우선 검토할 대상은 주소 공개와 발행 구조다. D1 대비 P의 추가 이득이 남아야 미래 예측기 자체의 가치를 따로 주장할 수 있다. 이 비교는 아직 실행하지 않았다.

04 Blind spot 2 P0

예측기 밖으로 새는 미래 정보

현재 후보 생성은 전체 request 수와 index로 미래 정답 page를 알고, 잘못된 후보 ID도 전체 request 수를 이용해 만든다. 더 중요한 부분은 wrong 표시가 통계에만 쓰이지 않고 ready 시 즉시 버퍼를 반환하는 제어에 쓰인다는 것이다. [R1 28행 66행 203행]

예측 정확도만 현실화해서는 부족한 이유

실제 controller는 page가 ready가 된 순간 그 page가 앞으로 영원히 사용되지 않을지 알 수 없다. 미래를 모르는 predictor를 붙여도 eviction·admission·종료 처리에서 정답을 읽으면 자원 비용이 계속 낙관적이다. 후보를 언제 버릴지는 관측된 압력, 나이, TTL, confidence 같은 현재 정보로 결정해야 한다.

권장 정보 경계

구성요소	읽을 수 있는 정보	금지할 입력
Workload 공급기	전체 trace와 예정 도착	정책 객체로 미래 목록 전달
Predictor와 scheduler	현재까지 공개된 요청·완료·자원 상태	미래 사용 여부와 oracle wrong
사후 평가기	완료된 로그와 전체 trace	제어 결정에 정답 피드백
온라인 feedback	실제 사용·만료·eviction 사건	아직 관측하지 않은 향후 사용

동일 prefix 테스트

시각 t까지 공개되는 입력이 같은 두 trace를 만든다. t 이후 주소, 길이, 도착 간격을 다르게 바꾸고 동일 정책 seed로 실행한다. t 이전의 후보·NAND issue·버퍼 반환·eviction·admission 결정 로그가 동일해야 한다. 외부 서비스 난수도 같은 입력 사건에 같은 값이 배정되도록 고정한다.

온라인 로그와 사후 useful 판정은 분리한다. 두 trace의 미래 사용이 달라 사후 useful 비율이 달라지는 것은 허용되지만, 그 차이로 과거의 폐기 시각이 바뀌면 실패다. 종료 시점 자체도 정책에 언제 알려지는지 계약에 넣는다.

고찰 질문 예측기가 볼 수 있는 신호 목록을 함수 인자로 제한할 수 있는가. 전체 trace 객체를 전달하지 않고도 모든 정책을 구현할 수 있는가.

05 Blind spot 3 P0

요청 예산을 하나의 숫자로 더하는 문제

AR credit 하나는 64B transaction 상태이고 page descriptor 하나는 최대 16KiB NAND 작업을 대표한다. 이를 같은 1개로 더하면 실행 제한은 만들 수 있지만 물리 자원 동등성은 만들 수 없다. 현재 same_total 결과는 혼합 토큰 감도 분석으로 남겨야 한다. [R1]

자원	최소 계수 단위	점유 종료 조건
Host credit	포트별 수락된 burst	마지막 R beat handshake
Lookup와 decision	각 pipeline의 활성 작업	해당 계산 완료
Page command entry	unique 물리 page와 version	command 추적이 끝나는 정의된 시점
Consumer metadata	page에 연결된 demand 참조	해당 참조의 데이터 반환
예약 버퍼	예약된 bytes	실제 저장/반환 정책에 따른 해제
Resident SRAM	실제로 존재하는 bytes	소비 후 release 또는 안전한 eviction
Predictor metadata	stream table과 history bytes	entry 교체 또는 stream 종료

동일 page 예산과 동일 비용은 다른 실험이다

같은 page 예산 비교는 unique page 읽기를 몇 개 겹칠 수 있는지 맞춘다. Demand-only page, speculative-only page, 양쪽이 합류한 page 집합을 서로 겹치지 않게 나누어 합산한다. 합류했다고 새 page entry나 새 NAND read를 추가하면 안 된다.

$$U = U_{demand} + U_{speculative} + U_{shared}$$

같은 구현 비용 비교에서는 각 entry의 실제 폭과 queue 깊이를 곱한다. Page 준비 후 speculative descriptor를 지워도 주소 tag, version, 참조 수 등 필요한 metadata가 사라지는 것은 아니다. 현재 Python 객체 크기를 RTL SRAM 면적으로 대신 사용하지 않는다.

$$M = N_{AR} \times d_{AR} + N_{page} \times d_{page} + N_{ref} \times d_{ref} + M_{predictor} + M_{data}$$

여기서 N은 entry 수, d는 entry당 bytes, M은 총 bytes다. AR entry를 32B로 가정하면 4포트 O502는 62.75KiB, O10000은 1,250KiB다. 이는 저장량 추정이며 RTL 면적 측정이 아니다. 포트·bank·ECC·배선·제어와 추가 page/predictor 비용을 포함해 같은 성능의 Pareto 경계를 찾아야 한다.

06 Blind spot 4 P0

병합과 재사용을 건디는 page 수명주기

현재 page ID는 요청 index에서 만든 고유 번호이고, 병합은 주로 같은 page의 구성 burst 사이에서 일어난다. 서로 다른 요청이 같은 물리 page를 다시 읽는 cache 수명주기는 구현되어 있지 않다. returned가 page의 전체 예상 burst 수에 도달하면 버퍼를 해제하는 규칙도 이 workload에 맞춰져 있다. [R1, R2]



제안 수명주기 demand 합류는 새 page 읽기가 아니라 consumer 추가다

그림 2 Page의 실행 상태와 consumer 참조를 분리하는 제안 수명주기

상황	필요한 동작	검증할 불변식
Demand와 prefetch가 같은 page 요청	하나의 canonical entry에 consumer 연결	동일 version의 중복 NAND issue 없음
READY 뒤 다른 요청이 같은 page 사용	상주 여부 확인 후 refcount 증가	해제된 저장 공간을 ready hit로 취급하지 않음
오류 후보가 끝까지 미사용	TTL 또는 압력에 따른 안전한 eviction	미래 정답으로 즉시 폐기하지 않음
쓰기나 주소 재매핑 발생	version 갱신 또는 invalidation	오래된 데이터가 새 consumer에 전달되지 않음
진행 중 작업 취소 요청	물리 취소 가능 단계까지만 취소	이미 사용한 NAND 시간과 bytes 보존

읽기 전용 1차 모델의 권장 범위

처음에는 주소 공간을 불변의 read-only로 고정하고 canonical physical page, consumer 목록, refcount, pin 상태를 구현하는 것이 검증하기 쉽다. Version·invalidation은 인터페이스에 남기되 쓰기와 FTL 변경은 명시적으로 범위 밖으로 두거나 별도 확장으로 추가한다.

최종 refcount가 0이라는 이유만으로 캐시 데이터를 반드시 지울 필요는 없다. 이후 재사용을 허용할지, 일회성 stream buffer로 바로 반환할지 정책을 고정해야 한다. 두 기준선에 같은 정책을 적용해야 cache 이득이 predictor 이득에 섞이지 않는다.

07 Blind spot 5 P1

정확도와 제때 준비되는 비율의 차이

현재 accuracy는 후보마다 Bernoulli로 정답 여부를 정하는 입력이다. 발행 제한·drop·대기 때문에 실제 발행된 후보의 precision은 이 값과 달라질 수 있다. 또한 최종 사용되는 page라도 너무 늦게 도착하면 critical path를 거의 줄이지 못한다. [R1]

지표	권장 분자와 분모	읽을 때의 주의점
Issued precision	최종 사용된 spec page / 발행 spec page	발행 전 drop은 별도 계수
Coverage	spec가 먼저 발행된 demand page / demand page	캐시 재사용과 중복 소비의 분모 고정
Timely coverage	정한 demand deadline 전 ready / demand page	AR 전 또는 lookup 완료 전 중 정의 필요
Late merge	미완료 spec에 demand가 합류한 수	늦어도 일부 지연 절감 가능
Request readiness	모든 구성 page가 제때 준비된 요청 / 요청	page 비율과 다름
Unused와 unresolved	미사용으로 종료된 수와 미판정 재고	관측 종료 시 미사용을 영구 오예측으로 단정 금지

90퍼센트의 두 가지 뜻

설명용으로 각 page가 제때 준비될 확률 q 가 0.90이고, 4개 page의 준비 여부가 서로 독립이라고 가정하자. 64KiB 요청 전체가 준비될 확률은 다음과 같다. 이는 precision이 90%라는 가정과 다르다.

$$P(\text{all 4 ready}) = q^4 = 0.9^4 = 0.6561$$

65.61%는 독립 가정의 이론값이다. 실제로는 같은 채널의 혼잡이나 연속된 예측 실패가 함께 발생할 수 있어 이 값도 달라진다. Page 평균만으로 요청 p99를 예측할 수 없다.

실제 predictor를 평가할 축

먼저 sequential 또는 stride 예측기를 과거 주소만으로 동작시킨다. Distance와 degree를 바꾸며 실제 precision·coverage·lead 분포를 얻는다. 합성 모델의 accuracy=1, lead=10us 조합이 실제 예측기로 동시에 달성 가능한지는 별도의 결과로 보고한다.

08 Blind spot 6 이론 추정

대역폭 여유가 요구하는 예측 품질

높은 outstanding에서 demand-only가 이미 256GB/s를 내면, 잘못된 prefetch는 유효 데이터와 채널을 경쟁한다. 아래 식은 재사용·중복 읽기·eviction 재읽기가 없고 모든 page 크기가 같다는 제한된 가정에서 얻는 필요조건이다. 충분조건은 아니다.

N개 demand page 중 cN개를 유효 speculative 읽기로 대체하고, 발행 precision을 p라 두면($p > 0$) 전체 spec 읽기는 cN/p 개다. 나머지 demand 읽기는 $(1-c)N$ 개이므로 NAND read amplification A는 다음과 같다. Late merge로 읽기가 하나인 경우도 유효 spec에 포함하되 timely 성능 이득은 따로 본다.

$$A = \frac{(1 - c)N + cN/p}{N} = 1 - c + \frac{c}{p}$$

$$B_{useful} \leq \min(B_R, \frac{C_{NAND}}{A})$$

현재 파라미터의 data transfer와 turnaround만 반영한 채널 상한은 다음과 같다. Command/data 공유 버스, sense 충돌, ECC 병목을 추가하면 실제 공급은 더 낮을 수 있다.

$$C_{NAND} = \frac{128 \times 16384}{16384/2400 + 0.05} / 1000 \approx 304.966 \text{ GB/s}$$

p c는 1	A	256GB/s에 필요한 NAND	채널만 고려한 가능성
1.00	1.000	256.000GB/s	필요조건 충족
0.90	1.111	284.444GB/s	여유가 줄어듦
0.84	1.190	304.762GB/s	거의 여유 없음
0.75	1.333	341.333GB/s	현재 상한 초과

$c=1$ 에서 256GB/s의 필요조건은 p가 약 0.8394 이상인 것이다. 이를 제품 최소 정확도 사양으로 바로 사용하면 안 된다. 재읽기·부분 page 활용·경합·tail 여유를 생략했고, c가 달라지면 경계도 달라진다.

고찰 질문 성능이 포화된 영역에서는 throughput 증가 대신 credit 절감이나 p99 개선이 비용을 상쇄하는가. 불필요 NAND bytes의 허용 상한을 제품 목표와 함께 정해야 한다.

09 Blind spot 7 P1

선행 시간이 길어질 때 숨는 버퍼 비용

Prefetch를 일찍 시작하면 demand를 기다리는 데이터가 많아진다. 너무 이르면 사용 전 eviction을 겪고, 너무 늦으면 demand stall을 줄이지 못한다. 따라서 lead는 클수록 좋은 값이 아니라 예측 품질·버퍼·혼잡과 함께 조절하는 값이다.

예약과 실제 상주를 따로 계산하기

현재는 NAND issue 전에 전체 page를 예약한다. Reservation은 미래 데이터를 받을 용량 보장이고 resident bytes는 이미 도착한 데이터다. 두 상태를 같은 byte로 이중 계산하지 않으면서 각각의 시간 적분과 최고값을 기록해야 한다. Predictor와 page metadata는 별도 저장 비용이다.

$$E[B_{unused}] \approx \lambda_{unused} \times P \times E[T_{resident}]$$

이론 예시로 spec 발행률 4page/us, 최종 미사용 비율 10%, page 크기 16KiB, 미사용 page의 resident 수명 100us를 가정하면 평균 655,360B, 즉 0.625MiB다. 안정된 입력·충분한 용량의 단순 체류 계산이며 실제 모델 측정값이나 보장된 peak는 아니다.

폐기 정책 후보	얻는 것	잃거나 검증할 것
고정 TTL	최대 상주 시간을 제한하기 쉬움	늦게 도착한 유용한 demand가 재읽기
압력 기반 eviction	사용 가능한 빈 공간을 활용	오류가 쌓인 뒤 demand 도착 시 지연
낮은 confidence 우선	예측 품질 정보를 활용	confidence 보정과 업데이트 비용
Spec 전용 부분 예산	Demand 저장 공간 보호	유휴 공간 활용률과 경계 마찰

반드시 넣을 반례

P를 prefetch한 뒤 TTL 직전에 demand가 합류하는 경우, READY page를 eviction하는 순간 다른 포트가 참조하는 경우, 사용 후 재사용하는 경우를 검사한다. Pin/refcount가 있는 데이터를 덮어쓰지 않아야 하고, expiry와 demand가 같은 시각이면 정한 event 순서대로 결정해야 한다.

현재 wrong-ready 즉시 폐기는 이 체류 비용을 평가하지 못한다. TTL과 압력 정책을 연결한 뒤 buffer peak뿐 아니라 byte·us 적분과 재읽기 bytes를 비교해야 한다.

10 Blind spot 8 PO 계약 확인

AXI burst 길이와 반환 순서의 영향

기존 64B burst는 512bit R 포트에서 1 beat다. 이 기준을 유지해야 기존 O502와 O10000을 재현할 수 있다. 다만 실제 제품이 긴 burst를 지원하면 credit 1개의 데이터 범위와 RLAST까지의 체류가 달라진다.

Burst bytes	64KiB 요청의 burst 수	16KiB page의 burst 수
64B	1,024	256
256B	256	64
4KiB	16	4

Arm AXI4 규격은 INCR burst를 최대 256 transfer까지 지원하며 burst가 4KiB 경계를 넘지 않도록 요구한다. 위 4KiB 예시는 4KiB 정렬, 512bit beat 64개, 해당 전송을 지원하는 인터페이스를 가정한다. 제품의 최대 burst·주소 정렬은 별도로 확인해야 한다. [E1 A3.4.1]

포트 전체 FIFO가 필요한지 확인하기

현재 R 반환은 포트 안에서 FIFO다. Arm 규격의 read ordering은 같은 ARID의 순서를 유지하며, 다른 ID 사이에는 순서 밖 반환이나 interleaving이 허용될 수 있다. 실제 구현이 이를 지원하는지는 별개다. 현재 FIFO는 제품 선택될 수 있지만 모든 AXI 구현의 필수 동작으로 일반화할 수 없다. [E1 A5.3]

앞의 느린 burst 때문에 뒤의 ready burst가 막히는 HOL을 predictor가 완화할 수 있다. 그러나 multi-ID 반환을 허용하는 것만으로 같은 효과가 나오면 이는 ordering 구조의 이득이다. FIFO와 ID별 FIFO를 별도 실험 축으로 두고 reorder metadata와 데이터 버퍼 비용도 계산한다.

포트와 요청의 대응

현재 한 host request의 burst가 네 포트에 round-robin으로 나뉜다. 독립 tenant가 포트마다 하나씩 쓰는 구조와 다르므로 이 workload에서 per-port 처리량이 비슷하다는 것만으로 QoS가 공정하다고 말할 수 없다. 요청 완료는 가장 늦은 구성 burst에 의해 결정된다. [R3]

권장 비교 먼저 기존 64B/FIFO 기준선을 잡는다. 이후 burst 길이·ID 수·port mapping 중 한 축씩만 바꾸어 predictor와 구조 변경의 기여를 분리한다.

11 Blind spot 9 P1

용량으로 드러나지 않는 물리 병목

2.5MiB 버퍼에서 유한 후반 256GB/s를 관측했다는 결과는 SRAM 용량의 모델 결과다. 그 SRAM이 실제로 같은 속도로 데이터를 쓰고 읽을 수 있다는 검증은 아니다. 모델에 없는 bandwidth나 bank 제약을 peak bytes로 대신 평가할 수 없다.

SRAM을 한 번 거치는 경우의 계산

유효 데이터 256GB/s를 SRAM에 한 번 쓰고 한 번 읽으면 데이터 이동은 쓰기 256GB/s와 읽기 256GB/s, 합산 512GB/s다. 듀얼 포트나 분리 bank 구조에서는 방향별 요구량을 따로 만족해야 한다. 잘못된 page, ECC, 재전송, 정렬 손실은 추가 비용이며 이 수치는 회로 측정값이 아니다.

자원	현재 모델에서 보이는 것	추가해야 할 제약
NAND channel	data transfer와 turnaround 점유	command/address가 같은 버스를 쓰는지
Chip과 plane	command busy와 plane busy	die/LUN, latch, cache read의 실제 overlap
SRAM	예약 byte 수	bank 수, 읽기/쓰기 port, arbitration
ECC	고정 지연	pipeline II, 동시 page 수, 오류별 service
R 경로	연속 반환 가능한 포트	RREADY stall, beat별 backpressure

Command와 data가 같은 채널을 쓰는 경우

현재 dispatch는 command의 chip busy와 transfer의 io busy를 별도로 검사한다. Command issue가 io_busy를 검사하지 않으므로 같은 channel에서 command와 data를 독립적으로 겹칠 수 있는 추상화다. 실제 NAND interface가 해당 자원을 공유한다면 command bus phase를 채널 점유에 포함해야 한다. 실제 제품 조건을 확인한 뒤 수정해야 할 예정이다. [R1 75행, R2]

NAND utilization도 data transfer+turnaround의 채널 이용률이다. Plane sense utilization이나 chip 전체 활용도와 같지 않다. 채널 100%가 아닌데 throughput이 낮다면 sense, lookup, HOL, 버퍼 예약 중 어떤 원인이 막았는지 계측해야 한다.

12 Blind spot 10 P1

판단 지연과 판단 처리량의 분리

현재 상속 경로에는 prefetch_decision_us와 host_decision_us의 고정 지연 옵션이 있다. 이를 아예 없다고 보아서는 안 된다. 부족한 부분은 실제 predictor의 decision queue, 동시 slot과 initiation interval을 제한하는 모델이다. Lookup은 이와 별도로 slot·clock·pipeline·II를 이미 갖는다. [R3]

$$\mu_{decision} = \frac{n \times f}{II_{cycles}}$$

$$N_{active} \approx \lambda_{decision} \times L_{decision}$$

n은 lane 수, f는 초당 clock 수, II는 lane당 새 작업 투입 간격의 cycle 수다. II를 초 단위로 쓰면 처리량은 n/II_seconds이며 f를 다시 곱하지 않는다. 평균 동시 작업 식은 안정된 상태의 근사다.

판단 단위	합계 64GB/s에서 입력률	합계 256GB/s에서 입력률
64KiB host request	0.9766M회/s	3.9063M회/s
16KiB page	3.9063M회/s	15.625M회/s
64B AR burst	1,000M회/s	4,000M회/s

한 요청마다 판단하는 회로와 매 burst마다 판단하는 회로는 위 구성에서 입력률이 1,024배 다르다. 포트당 16GB/s라면 64B burst 입력 간격은 4ns다. 100ns 판단을 완전 pipeline으로 처리하려면 평균 약 25개 활성 slot과 4ns 이하의 유효 II가 필요하다는 이론 근사다.

같은 timestamp의 처리 순서도 모델 계약이다

현재 event queue는 시간 뒤에 삽입 순번을 사용한다. 같은 시각의 demand 공개, prediction, buffer 반환, arbitration 이 어느 순서로 실행되는지에 따라 drop과 즉시 발행 여부가 달라질 수 있다. 순서를 문서화하고 동시 사건을 일괄 반영한 뒤 중재하는 방식도 별도로 검토한다.

필수 검사 demand 도착과 spec expiry, RLAST와 새 AR, NAND 완료와 demand promotion이 같은 시각인 microtrace를 만든다. Event 생성 순서만 바꾸어도 결과가 달라지면 그 차이가 의도된 하드웨어 순서인지 설명할 수 있어야 한다.

13 Blind spot 11 P1

Demand 우선권과 prefetch 기아의 균형

Demand를 먼저 고르는 정책은 이미 실행 중인 spec sense나 transfer를 없애지 못한다. Spec가 버퍼·plane을 선점한 뒤 demand가 도착하면 strict priority여도 demand가 기다린다. 또한 demand가 계속 있으면 새로운 spec는 무한히 대기할 수 있다.

현재는 command와 transfer에서 demand를 우선하고, throttle은 queued demand가 있거나 host active가 총 credit의 80% 이상이면 spec issue를 멈춘다. Accuracy 0.9·lead 10us·limit 128의 짧은 64GB/s 입력 비교에서 전체 처리량은 26.346에서 14.727GB/s로 감소했고 wrong page 수는 26에서 15로 줄었다. 이는 특정 합성 비교 결과이며 이 정책의 일반 우수성을 지지하지 않는다. [R4]

정책 후보	관측하는 압력	함께 볼 비용
Strict demand first	대기 중 demand	spec 미발행 age와 회복 지연
Demand reserve와 spec cap	예약 bytes와 unique pages	보호 공간 때문에 생기는 유희 시간
채널별 cap와 hysteresis	채널 queue age와 이용률	동작 진동과 hotspot 대응
제한적 aging 또는 quota	spec 대기 시간과 유효성	demand p99와 서비스 공백

80퍼센트 하나로 병목을 판단하기 어려운 이유

AR credit 점유는 NAND 혼잡뿐 아니라 lookup 대기, RREADY stall, FIFO HOL로도 높아진다. Credit 압력만으로 prefetch를 끄면 지연을 가릴 기회까지 잃을 수 있다. 채널별 demand age와 버퍼 부족 원인, 실제 useful feedback을 분리하는 것이 좋다.

Feedback Directed Prefetching 연구는 accuracy·timeliness·cache pollution을 함께 사용해 aggressiveness를 조정하는 접근을 제시했다. 여기서는 정책 설계의 참고 근거로만 사용한다. CPU cache의 실험 성능 수치를 NAND 모델로 옮기지 않는다. [E2]

제안 실험 부하를 낮음→포화→낮음으로 전환하고 잘못된 예측을 특정 채널에 집중시킨다. 감소한 낭비 bytes, 늘어난 demand p99, spec 재개 시간, 최대 무서비스 시간을 한 묶음으로 비교한다. Threshold 수치는 튜닝 전의 설계 변수로 남긴다.

14 Blind spot 12 P1

후반 처리량과 정상 운영 상태의 차이

새 모델의 initial/middle/late는 완료 burst 수로 자른 0~25%, 25~75%, 75~100% 구간이다. 길이가 같은 wall-clock 창이 아니며 마지막 구간에는 이미 준비된 데이터의 소진과 공급 종료 효과가 섞일 수 있다. 기존 tail은 후반 50%이므로 나란히 쓸 때 정의를 표시한다. [R1, R3]

관측 창	시작과 끝	함께 기록할 재고
Cold 전체	첫 요청 도착부터 마지막 demand 완료	AR 전 backlog와 학습 비용
운영 공통 창	모든 비교군에 동일한 t0와 t1	시작/끝 queue, buffer, spec 재고
Whole job	최초 허용 준비부터 최종 drain	미완료 spec와 실제 소비 자원

$$Q(t_1) - Q(t_0) = A(t_0, t_1) - D(t_0, t_1)$$

A와 D는 같은 단위로 센 도착과 완료 수다. 처리량이 입력률보다 낮고 queue가 계속 증가하면 유한 실행 p99를 지속 운영의 지연으로 제시할 수 없다. 입력 64GB/s에서 후반이 잠시 64GB/s를 넘는 것은 backlog를 해소했기 때문일 수 있다.

실행 길이와 p99 표본 수

010000·512요청 demand 실행의 전체 host 구간은 약 140.829us다. 32MiB로 늘렸다는 사실만으로 장시간 정상상태가 증명되지는 않는다. 요청 64개의 nearest-rank p99는 최댓값이며, 512개에서는 여섯 번째로 큰 값이다. 작은 표본과 deterministic service의 tail을 실제 제품 p99로 일반화하지 않는다.

길이와 난수를 통제하는 방법

제안은 512→2,048→8,192요청으로 늘려 동일 시간창의 throughput·backlog 기울기·buffer 적분을 비교하는 것이다. 아직 실행하지 않았다. 변동 지연을 추가하면 seed 반복과 run별 결과 범위를 남긴다. 동일 seed뿐 아니라 service-time 표본을 page/visit 같은 안정된 key로 대응시켜 정책별 난수 소비 순서의 영향을 피한다.

Open-loop는 예정된 시각에 계속 공급하고, closed-loop는 완료에 따라 다음 요청을 낸다. 후자는 prefetch가 도착 trace 자체를 바꾸므로 고정 trace replay와 답하는 질문이 다르다.

15 Blind spot 13 P1

Workload와 오류 분포가 바꾸는 경계

현재 핵심 결과는 순차 고유 page의 균등 striping을 사용한다. 이는 지연을 겹칠 기회와 채널 병렬성을 보기 좋지만, random·hotspot·reuse·부분 page 사용에서 같은 경계가 유지된다는 근거는 아니다. Workload 확장은 성능 범위를 넓히기보다 가정이 어디서 깨지는지 찾는 실험으로 시작한다.

Trace 유형	검증하는 반대 가설	함께 고정할 조건
순차 cold stream	기존 이득이 재현되는가	동일 초기 상태와 주소 공개
균등 random	실제 predictor가 lead를 만들 수 있는가	주소 수와 page mapping
Hotspot과 reuse	cache가 predictor 이득을 설명하는가	동일 cache 용량과 eviction
여러 stream 혼합	다른 stream이 history를 오염시키는가	stream ID 공개와 배분
부하와 stride 전환	학습과 throttle 회복이 늦어지는가	전환 시각과 입력 bytes
채널 편중	평균 이용률이 hotspot 병목을 숨기는가	정해진 physical mapping
부분 page 읽기	NAND bytes가 유효 bytes보다 커지는가	요청 크기와 page 내 locality

짧은 요청은 별도의 읽기 증폭을 만든다

16KiB page를 읽고 그중 64B만 사용하며 재사용이 전혀 없다면 NAND bytes/유효 bytes는 256이다. 이는 예측 오류가 없는 경우에도 생기는 granularity 비용의 이론 예시다. 현재 64KiB 전량 소비 결과에 붙인 bandwidth 경계를 이런 workload에 그대로 적용할 수 없다.

같은 평균 정확도의 다른 결과

정답 후보 비율 90%라도 오류를 균등하게 흩뿌린 trace와 10% 구간에 몰아넣은 trace는 연속 stall과 버퍼 압력이 다르다. 같은 비율을 유지한 paired 오류 주입과 특정 channel 집중을 비교해야 한다. 이 오류 주입은 합성 sensitivity 도구이며 실제 predictor 평가와 별도로 표시한다.

Runtime을 줄이려면 먼저 4~16요청의 의도적인 반례로 병합·매핑·정책을 검증하고, 결과가 설명되는 trace만 길이를 늘린다. 비교군마다 유리한 주소 mapping이나 cache 초기 상태를 선택하지 않는다.

16 모델 발전을 위한 지표 정의

추가 계측과 결과 해석 계약

Counter를 많이 추가하기 전에 각 지표의 단위·시작·끝·분모를 고정한다. 평균값만 저장하면 원인을 되짚기 어렵다. 작은 smoke에는 원시 trace를 남기고 큰 실행에는 bounded histogram과 자원 적분을 사용한다.

대상	최소 기록	해석 목적
Host request	arrival, first AR, first data, last RLAST	AR 전 대기과 전체 완료 지연
AR burst	accept, decision, lookup, R start, RLAST	credit 체류와 단계별 대기
Page	predict, demand_seen, issue, ready, release	예측 lead와 demand 이후 추가 대기
Consumer	attach, first/last return, detach	중복 병합과 정확히 한 번 완료
Spec outcome	issued, timely, late, unused, unresolved	정확도와 시간 적합성 분리
Channel과 plane	busy 적분, queue age, stall 이유	실제 공유 병목과 편향
Buffer와 metadata	reserved/resident bytes와 적분	peak 용량과 체류 비용
Stream과 port	공통 창 bytes, p99, 무서비스 시간	fairness와 starvation

대기 시간의 기준점을 고치기

현재 demand_max_nand_queue_us는 page 생성부터 issue까지의 age다. Spec 후보가 나중에 demand로 승격되면 demand가 존재하기 전의 대기도 포함된다. 새로 demand_seen을 기록하고 issue-demand_seen을 별도로 계산해야 순수 demand queue 대기를 볼 수 있다. [R1 118행]

기여도와 실제 사용을 구분하기

Demand가 이미 발행한 page에 predictor가 뒤늦게 같은 주소를 낸 경우, 이를 prefetch 성공으로 세지 않는다. 실제 latency 절감은 동일 조건의 기준선과 짝 비교한다. 한 실행의 ready hit만으로 절감 시간을 확정하지 않는다.

Whole job 비용과 에너지

총 NAND bytes와 command 수, SRAM read/write bytes, buffer byte·us, decision 횟수를 최종 drain까지 합산한다. 에너지는 실제 단위당 계수 e가 있을 때만 합산한다. 계수가 없으면 위 활동량을 비용 대리지표로 제시하고 pJ/B나 전력 수치를 만들지 않는다.

17 후속 DOE 제안 미실행

작은 실험으로 모델의 방향을 정하기

모든 축을 한 번에 sweep하면 해석과 실행 비용이 모두 커진다. 아래는 필요한 기능을 구현한 뒤 적용할 단계별 예시다. 조합 수는 계산한 값이고 runtime은 새 smoke 측정 전에는 미정이다.

단계	실험 설계	조합 수
A 계약 확인	2개 microtrace × 4모드 D0 D1 P R	8
B 핵심 경계	4개 O × 2개 buffer × 2개 공급 방식 × 3모드	48
C 정책 감도	2 workload × 2 pressure × 3정책 × 2 predictor 설정 × 3 seed	72
D 길이 확인	6개 선택 조건 × 3개 길이	18

위 예시는 합계 146회다. B의 O 예시는 128·502·2,000·10,000이며, buffer와 공급률을 고정한 뒤 관측된 경계 주변에 6,000·8,000 등 지점을 추가한다. Ready는 A에서 이상적 참고치로 유지하고, B는 D0·D1·P의 원인 분리에 집중한다. 세부 조건은 제품 목표에 맞춰 확정한다.

시간과 저장 공간을 먼저 추정하기

$$T_{estimate} = \sum n_k \times median(t_k)$$

k는 요청 수·정책·자원 크기가 비슷한 실행 계층이다. 계층마다 짧은 실행을 측정해 runtime, event 수, 최대 RSS, 결과 파일 크기를 기록한다. 큐 정렬·히스토리 때문에 요청 수에 선형 비례하지 않을 수 있으므로 큰 길이에서 확인용 실행을 한 번 더 둔다. 기존 413.75초 합계를 새 모델 runtime으로 복사하지 않는다.

경계 주변에서 답할 질문

64GB/s에서는 동일 입력률을 유지하면서 O와 버퍼를 어디까지 줄일 수 있는지 본다. 포화 공급에서는 common-window 처리량과 p99가 어떤 자원에서 더 이상 개선되지 않는지 본다. 같은 O 비교와 같은 unique-page/byte 예산 비교는 서로 다른 plot으로 만든다.

각 조건은 measured, failed, timed_out, not_run 중 하나로 기록한다. 중단된 조건을 성공한 평균으로 채우지 않는다. Config, seed, source hash, workload hash가 같은 경우에만 resume한다. 현재 runner의 case 이름만 확인하는 resume 방식은 후속 개선 대상이다. [R8]

18 후속 테스트 제안 미실행

반례 중심의 검증 목록

현재 56개 테스트를 회귀 기준으로 보존하되, 아래 항목을 새 모델의 통과 조건으로 추가한다. 성능 숫자가 좋아져도 보존식이나 인과성 검사가 실패하면 해당 결과는 설계 근거로 사용하지 않는다.

검증 대상	작은 반례 입력	통과 조건
Credit 보존	O1과 RREADY stall	active = accepted - completed, 한도 준수
동일 page 병합	4포트가 같은 page 동시 요청	read 1회, consumer 모두 1회 완료
Demand 승격	spec queued/sense/ready마다 합류	중복 발행과 이중 비용 없음
재사용	사용→반환→같은 page 재요청	resident 여부에 맞는 hit 또는 새 read
TTL과 pin	expiry와 참조 합류 동시 발생	참조 중 데이터 해제 없음
인과적 prefix	같은 과거와 다른 미래 trace	공개 경계 전 제어 로그 일치
오류 후보	accuracy 0 상당의 미사용 후보	예산 준수, 미래 정답 없는 폐기
작은 버퍼	모든 포트 head miss와 spec full	진행 보장 또는 명확한 모델 오류
Ordering	같은 ID와 다른 ID 반환	선언된 ID별 순서 계약 준수
Spec 기아	지속 demand와 뒤이은 idle	대기·미발행 기록, 재개 동작 일치
Cancel와 drain	발행 전후 취소 및 입력 종료	발생한 작업 비용 보존, 정확한 반환
동시 event	완료·AR·expire 동일 시각	선언한 tie-break 규칙으로 재현

자원마다 보존식을 둔다

$$allocated - released = active \geq 0$$

AR credit, page command entry, consumer 참조, 예약 bytes마다 별도의 ledger를 둔다. Cache를 종료 때 남기는 정책이라면 active=0을 강제하지 말고 최종 resident 재고를 별도로 설명한다. 물리 작업 drain과 cache flush는 같은 의미가 아니다.

Starvation 방지는 finite test 통과만으로 증명할 수 없다. Demand에 최소 서비스 기회를 주는 중재 가정, admission 제한, 최악의 자원 의존 관계를 분석하고 adversarial trace로 보완한다. strict demand priority에서 spec 기아는 허용된 정책 결과일 수도 있다.

19 권장 발전 계획

고찰을 구현 결정으로 바꾸는 순서

실제 predictor를 먼저 복잡하게 만드는 것보다, 정보 경계와 자원 소유권을 먼저 고정하는 편이 이후 결과를 해석하기 쉽다. 아래 단계는 구현 계획이며 이 문서 작성 중 simulator를 변경하거나 새 sweep을 실행하지 않았다.

단계	산출물	다음 단계로 넘어갈 기준
1 계약	주소 공개, D0/D1/P 정의, 자원 vector	같은 정보와 같은 비용의 의미가 명확함
2 수명주기	canonical page, consumer, pin, eviction	병합·반환·재사용 반례 테스트 통과
3 인과적 제어	predictor API, feedback, decision 용량	prefix 검사와 미래 정답 차단 통과
4 물리와 정책	채널 phase, reserve, cap, throttle	demand 지연과 spec 기아를 설명 가능
5 계측과 DOE	공동 시간창, backlog, 예산별 비교	긴 실행과 paired 비교로 경계 재확인

설계자가 먼저 답할 질문

1. Controller가 처음 아는 것은 64KiB 요청 전체 주소인가, 수락된 64B AR인가. 공개 시점은 언제인가?
2. 1차 목표는 합계 64GB/s 유지인가, 256GB/s 포화인가. 요청 p99 허용값은 얼마인가?
3. O를 줄여 절감하려는 대상은 AR queue SRAM인가, lookup 자원인가, 외부 fabric 제한인가?
4. 읽기 전용 cache를 허용하는가. 미사용 page의 TTL과 eviction 기준은 무엇인가?
5. 제품의 burst 길이·AXI ID 수·RREADY stall·NAND command/data 공유 계약은 무엇인가?
6. 불필요 NAND bytes, buffer peak, demand p99 악화를 각각 얼마까지 허용하는가?

고찰 메모를 남기는 형식

각 선택마다 가정, 선택 이유, 반대 가설, 최소 반증 실험, 통과 기준을 한 문단씩 남긴다. 예를 들어 “O502의 이득이 예측 때문”이라는 가정에는 D1 조기 demand 발행이 같은 이득을 낼 수 있다는 반대 가설과 D0/D1/P 짝 비교를 붙인다.

제품 사양이 정해지기 전에는 임의의 숫자를 합격선으로 고정하지 않는다. 먼저 throughput·p99·wasted bytes·metadata·buffer의 곡선을 보고, 실제 제한을 넣어 선택 가능한 조합을 좁힌다.

20 근거 목록과 수정 위치

코드 근거와 재현 자료

코드 검토 기준은 4d3bb3f0eba163f0b44dd193b1db5b65958bf739다. 아래 링크는 고정 커밋을 가리킨다. 측정 기준 bf63379 재현 결과와 합성 확장 결과를 구분하며, 이 문서는 그 이후의 신규 성능 측정 결과를 포함하지 않는다.

R1 [deep_prefetch_sim.py](#) 합성 후보와 자원 및 측정

R2 [axi_burst_sim.py](#) page identity와 버퍼 및 NAND dispatch

R3 [multiport_sim.py](#) 포트 AR R 반환과 판단 지연

R4 [sweep.csv](#) 103조건의 측정 원자료

R5 [configs.json](#) 조건별 전체 설정

R6 [STATUS_KO.md](#) 기존 검토 정정과 미완료 항목

R7 [test_deep_prefetch.py](#) 추가된 6개 테스트

R8 [run_deep.py](#) 단계별 실행과 resume 동작

변경 주제	현재 주요 위치	후속 수정 방향
미래 정보	deep 28~74행	trace와 predictor 객체 분리
Oracle 폐기	deep 203~210행	관측 기반 TTL와 eviction
혼합 토큰	deep 50~74행 108행	AR/page/byte ledger 분리
시간창과 drain	deep 118~140행	whole-job와 공통 시간창 추가
수명주기	axi page와 deep rlast	canonical address와 consumer
Decision 처리 용량	multiport event와 AR 처리	지연에 II·slot·queue 추가

현재 단일 조건 재실행 예시 저장소 루트에서 다음 명령을 사용한다. case의 online은 합성 모델이라는 이름을 유지한다. 전체 sweep은 기존 결과 파일을 갱신하므로 별도 revision 출력 디렉터리를 쓰는 방식으로 발전시키는 것이 좋다.

```
python3 experiments/replay_deep_case.py
--case host_1.024_502_online_False
--out scratch/blind_spot_replay
```

위 명령은 한 줄로 이어 실행한다. 이 문서 작성 중에는 실행하지 않았다.

21 외부 문헌과 기존 검토의 관계

참고 문헌과 근거의 한계

제공받은 설계검토 문서

NAND_Prefetch_심화_설계검토.docx 및 동일 내용의 PDF를 검토 자료로 사용했다. 해당 문서는 bf63379 기준으로 인과적 predictor, page lifecycle, 같은 자원 비교, 단계별 DOE를 제안한다. 문서에 적힌 구현 지시는 이 요청에서 실행한 작업으로 취급하지 않았다.

이번 상세 검토는 주소 공개 시점에 따른 D1 기준선, oracle 폐기의 비용 왜곡, 혼합 단위 예산, p99 표본과 시간창, SRAM bandwidth, command/data 공유, event 동시성처럼 결과 해석을 바꿀 수 있는 부분을 구체화했다. 기존 문서를 대체하는 사양서가 아니라 후속 설계 결정을 위한 보충 자료다.

외부 1차 자료

E1 Arm IHI 0022E AXI and ACE Protocol Specification

2013년 판. A3.4.1, A3-46의 burst 길이와 4KiB boundary, A5.3, A5-80의 read ordering을 확인했다. 제품이 채택한 규격 판본과 지원 기능은 별도로 확인해야 한다.

E2 Srinath 등 Feedback Directed Prefetching HPCA 2007

Accuracy·timeliness·pollution을 함께 관측하는 정책 접근의 근거다. 본 문서의 NAND 성능이나 최적 threshold를 입증하는 자료로 사용하지 않았다.

E3 gem5 Prefetcher.py 고정 revision의 queue 설정 참고

Queue size, latency, queue squash/filter와 throttle 설정이 분리되어 있는 구현 참고다. 이 코드를 NAND 모델에 연결하거나 동일 성능을 검증한 것은 아니다. 실제 인터페이스는 제품 요구에 맞춰 설계한다.

확인 수준을 유지하기 위한 기록

이 문서의 계산은 명시한 단순 가정 아래의 필요조건과 설명 예시다. 실험 runtime·면적·전력·실제 NAND latency 분포를 새로 측정하지 않았다. 기존 결과는 CSV에서 읽었고, 수치 비교와 문서 렌더링을 검증했다. 외부 문헌 확인일은 2026년 10월 6일이다.

CSV SHA256 6c9ef99e79a74708ff47967781a2e2a64635fd288de002267ae7d75ba5077b39

모델 발전의 첫 판단은 “실제로 알고 있는 정보만으로, 같은 자원을 쓰면서, demand의 지연과 추가 작업 비용까지 포함해 이득이 남는가”이다. 이 질문에 답하는 데 필요한 계약과 반례를 먼저 고정하면 이후 예측기와 정책의 개선을 신뢰할 수 있다.